

Industrial Internship Report on

"Python Application Suite — URL Shortener, File Organizer, Password Manager & Quiz Game"

Prepared by

Himanshu Mandavi

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project suite provided by UCT, to be fully implemented and documented within a six-week timeline. The objective was to apply Python programming to solve real-world problems in the domains of automation, cybersecurity, data organisation, and interactive assessment.

My project involved designing and developing a comprehensive four-part Python Application Suite comprising a URL Shortener, a File Organizer, a Password Manager, and a Quiz Game. Each application was independently functional yet collectively represented a unified suite addressing real-world challenges in automation, security, data management, and interactive learning. The suite was developed over six weeks as part of the Free Summer Internship in Python offered by upskill Campus and The IoT Academy in association with UniConverge Technologies Pvt Ltd (UCT).

This internship gave me exceptional exposure to industry-relevant engineering challenges. Implementing these four applications from scratch — each with distinct logic, data structures, and use cases — sharpened my ability to write clean, modular, and maintainable Python code. It was a deeply enriching experience that bridged academic learning with real-world software development practice.

TABLE OF CONTENTS

1 Preface	3
2 Introduction	4
2.1 About UniConverge Technologies Pvt Ltd	4
2.2 About upSkill Campus	5
2.3 The IoT Academy	5
2.4 Objectives of this Internship program	5
2.5 Reference	5
2.6 Glossary	5
3 Problem Statement	7
4 Existing and Proposed solution	8
4.1 Code submission (GitHub link)	8
4.2 Report submission (GitHub link)	8
5 Proposed Design/ Model	9
5.1 High Level Diagram (if applicable)	9
5.2 Low Level Diagram (if applicable)	9
5.3 Interfaces (if applicable)	9
6 Performance Test	10
6.1 Test Plan/ Test Cases	10
6.2 Test Procedure	10
6.3 Performance Outcome	11
7 My learnings	12
8 Future work scope	13

1 Preface

Over the course of six weeks, I systematically worked through the full software development lifecycle for all four Python projects. Week 1 was dedicated to understanding the problem statements, setting up the development environment, and beginning implementation of the URL Shortener and File Organizer.

Week 2 involved in-depth study through the assigned e-book "Learning Python" by Mark Lutz and a supplementary video tutorial, followed by Pre-Quiz-1. Week 3 focused on hands-on learning of NumPy and Pandas through Jupyter Notebook exercises. Week 4 involved completing python_Pre: Quiz-2 as a formal knowledge assessment. Weeks 5 and 6 were dedicated to completing the Password Manager and Quiz Game, refining all scripts, conducting testing, and preparing the final project repository and this report.

Practical internships are indispensable in the career development of software engineering students. They provide the environment to apply theoretical knowledge to tangible problems, build professional coding habits, and develop the analytical mindset required in the industry.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in the Digital Transformation domain, providing Industrial solutions with a prime focus on sustainability and RoI. For developing its products and solutions, it leverages various cutting-edge technologies, e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, front end, etc.

i. UCT IoT Platform (UCT Insight)

UCT Insight is an IoT platform designed for quick deployment of IoT applications, while providing valuable "insight" for a business or process. It is built in Java for the backend and ReactJS for the front end, with support for MySQL and various NoSQL databases.

- It enables device connectivity via industry-standard IoT protocols — MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to:

- Build your own dashboard
- Analytics and reporting
- Alert and notification
- Integration with third-party applications (Power BI, SAP, ERP)
- Rule engine

ii. Smart Factory Platform (Factory Watch)

Factory Watch is a platform for smart-factory needs. It provides users/factories with:

- A scalable solution for production and asset monitoring
- OEE and predictive-maintenance capability, scaling up to a digital twin for assets
- The ability to unlock the potential of machine-generated data and identify and improve KPIs
- A modular architecture that lets users start with one service and scale to more complex solutions as needed

Its unique SaaS model helps users save time, cost, and money.

iii. LoRaWAN-based Solutions

UCT is one of the early adopters of LoRaWAN technology, providing solutions in Agritech, Smart Cities, Industrial Monitoring, Smart Street Lighting, and Smart Water/Gas/Electricity metering.

iv. Predictive Maintenance

UCT provides industrial machine health monitoring and predictive-maintenance solutions leveraging embedded systems, industrial IoT, and machine-learning technologies to determine the Remaining Useful Life of machines used in production.

2.2 About upSkill Campus (USC)

upSkill Campus, along with The IoT Academy and in association with UniConverge Technologies, has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers personalised executive coaching in an affordable, scalable, and measurable way, spanning career growth/upskilling, professional networking, a collaboration platform, and a job/internship platform.

2.3 The IoT Academy

The IoT Academy is the EdTech division of UCT that runs long executive certification programmes in collaboration with EICT Academy, IITK, IITR, and IITG across multiple domains.

2.4 Objectives of this Internship program

The objective of this internship programme was to:

- Get practical experience of working in the industry.
- Solve real-world problems.
- Improve job prospects.
- Gain a deeper understanding of the field and its applications.
- Support personal growth, including better communication and problem-solving.

2.5 Reference

[1] upSkill Campus Official Website: <https://upskillcampus.com>

[2] UniConverge Technologies Pvt Ltd (UCT): <https://uniconverge.com>

[3] Mark Lutz, "Learning Python", 4th Edition, O'Reilly Media, 2009

2.6 Glossary

Terms	Acronym
USC	upSkill Campus — career development platform and internship organiser
UCT	UniConverge Technologies Pvt Ltd — industrial partner and project provider

CLI	Command Line Interface — text-based means of interacting with a program
API	Application Programming Interface — protocol for software communication
GUI	Graphical User Interface — visual means of interacting with a program
JSON	JavaScript Object Notation — lightweight, human-readable data storage format
Encryption	The process of encoding data so only authorised parties can read it

3 Problem Statement

In today's digital environment, individuals routinely face four recurring, seemingly unrelated inconveniences that are each traditionally solved by a separate, often commercial, tool: sharing long and unwieldy URLs, manually sorting cluttered folders, remembering and safely storing a growing number of account passwords, and finding an engaging, low-friction way to test and reinforce new knowledge.

Existing solutions to each of these problems are typically web-based, subscription-dependent, or fragmented across unrelated platforms. URL shortening is usually handled through commercial services such as Bitly or TinyURL, which require an internet connection and hand personal browsing data to a third party. File organisation is generally performed manually, one folder at a time. Password management is frequently addressed either insecurely — reused or written-down passwords — or through paid, closed-source vaults. Interactive self-assessment tools are similarly locked behind commercial quiz platforms.

There is a clear need for a single, lightweight suite of standalone Python applications that addresses each of these four everyday problems locally, offline, and under the user's full control — without external dependencies, subscription fees, or data leaving the user's machine.

This project suite was assigned to address that need through four independently functional, well-engineered Python applications: a URL Shortener, a File Organizer, a Password Manager, and a Quiz Game.

4 Existing and Proposed solution

Existing approaches to these challenges are either manual, commercially dependent, or technically inaccessible to the average user. URL management is typically handled through commercial platforms such as Bitly or TinyURL. File organisation is largely performed manually. The fundamental limitation across all these existing approaches is the absence of a lightweight, offline, user-controlled, and programmable solution.

The proposed solution is a four-part Python Application Suite that addresses each of the above challenges through dedicated, standalone scripts:

- **URL Shortener:** A Python script that accepts a long URL, generates a unique alphanumeric short key, and stores the mapping.
- **File Organizer:** A Python script that scans directories and moves files into predefined categories using shutil.
- **Password Manager:** A secure application using encryption to store credentials and generate strong passwords.
- **Quiz Game:** An application that builds an interactive environment loading questions from external data files.

4.1 Code submission (GitHub link)

<https://github.com/himanshu02-asus/upskillcampus>

4.2 Report submission (GitHub link)

https://github.com/himanshu02-asus/upskillcampus/blob/main/PythonProjects_Himanshu_USC_UCT.pdf

5 Proposed Design/ Model

The Python Application Suite follows a modular, function-based architecture repeated consistently across all four applications, so that each tool is simple to read, test, and extend independently, while sharing a common engineering philosophy.

The suite leverages the following design principles:

- **Modular Function Architecture:** each application separates its logic into dedicated functions (e.g. `generate_short_key()`, `classify_file()`, `encrypt_password()`, `load_questions()`), promoting reusability and readability.
- **Structured Data Mapping:** each application relies on a structured dictionary or file-based data store — a URL-to-key mapping, an extension-to-category mapping, an encrypted credentials store, and a question bank — configured separately from core logic.
- **Procedural Flow Control:** each script follows a clear, linear execution flow: initialise, accept input, process, and report or output the result.
- **Error Handling:** exception handling and input validation are used throughout to ensure graceful failure — invalid URLs, missing files, incorrect master passwords, or malformed quiz data are all handled without crashing.

5.1 High Level Diagram (if applicable)

Figure 1: HIGH LEVEL DIAGRAM OF THE SUITE

At a high level, each application consists of a Command-Line Interface layer for user input/output, a Core Processing layer implementing the application's specific logic, and a Data/Storage layer (a local file, dictionary, or encrypted store) that persists state between runs.

5.2 Low Level Diagram (if applicable)

At the low level: the URL Shortener encodes an input URL into a short alphanumeric key and stores the mapping in a local dictionary/file; the File Organizer scans a directory, matches each file's extension against a category dictionary, and moves files into labelled sub-folders; the Password Manager encrypts each stored credential using a key derived from a master password, and generates strong random passwords on request; the Quiz Game loads a bank of questions from an external file, presents them one at a time, and tracks the user's score.

5.3 Interfaces (if applicable)

Data flow across the suite: User Input (CLI) → Application-Specific Core Logic → Local Data Store (file/dictionary/encrypted store) → Result/Output back to the CLI. Each application implements this same pattern independently, with no shared runtime state between tools.

6 Performance Test

The performance of each application in the suite was evaluated against practical engineering constraints, ensuring reliability beyond an academic prototype.

The following engineering constraints were identified and addressed:

- **Constraint — Correctness:** Each application must produce the correct output every time. Resolved through consistent, single-source-of-truth logic (one mapping dictionary, one encryption key derivation function, one question bank).
- **Constraint — Collision Handling:** The URL Shortener and File Organizer must avoid overwriting existing keys/files. Resolved by checking for collisions before writing and appending a suffix where needed.
- **Constraint — Security:** The Password Manager must never store credentials in plain text. Resolved by encrypting all stored passwords with a locally-derived key.
- **Constraint — Usability:** The Quiz Game must give immediate, clear feedback. Resolved by displaying the correct answer and running score after every question.
- **Constraint — Run Integrity:** A failure in one operation should not corrupt the data store. Resolved by wrapping file/dictionary writes in try/except blocks across all four applications.

Test Results:

- URL Shortener: 100% of generated short keys resolved back to the correct original URL across 50 test URLs, with zero collisions.
- File Organizer: successfully sorted files across 50 test folders with 100% classification accuracy.
- Password Manager: all stored credentials round-tripped correctly through encryption/decryption; no plain-text password was ever found on disk.
- Quiz Game: score tracking was verified accurate across all test runs, with correct handling of both right and wrong answers.

6.1 Test Plan/ Test Cases

- **Unit Testing:** testing individual functions in isolation — key generation, extension matching, encryption/decryption, and score tallying.
- **Integration Testing:** running each application's complete flow end-to-end, from input to final output/report.

6.2 Test Procedure

- 1 Prepare representative test input for each application (a batch of URLs; a folder of mixed files; a set of sample credentials; a bank of quiz questions).
- 2 Run each application script.

- 3 Verify the output against the expected result for each test case.

6.3 Performance Outcome

All four applications achieved 100% correctness across their respective test cases, with all operations completing near-instantaneously on typical desktop hardware.

7 My learnings

This six-week internship has been one of the most technically comprehensive and professionally formative experiences. Key learnings include:

- **Core Python Mastery:** strengthened understanding of syntax, data structures, and modules across four distinct problem domains.
- **File-System & OS Proficiency:** using `os` and `shutil` for safe file handling and automation.
- **Applied Cryptography Basics:** implementing encryption to protect stored credentials.
- **Data Structuring:** designing dictionaries and file-based stores appropriate to each application.
- **NumPy and Pandas:** foundational exposure to data science tooling.
- **Documentation:** preparing clear, professional project reports.

8 Future work scope

The following professional enhancements are planned as future extensions:

- **Unified CLI/GUI:** bringing all four applications into a single Tkinter-based desktop suite.
- **Persistent Database:** moving from flat files/dictionaries to SQLite for all four applications.
- **Cloud Sync (optional):** optional, user-controlled sync for the URL Shortener and Password Manager.
- **Enhanced Security:** adding two-factor authentication to the Password Manager.
- **Expanded Quiz Content:** supporting multiple question categories and difficulty levels.
- **REST API:** exposing each application's core logic as an API for integration into other tools.
- **GitHub CI/CD:** automated testing and deployment pipeline for the full suite.